



CAIO YURI LIMA
EDUARDO ZANETTI LUIS
ISABELA TRÓPICO PUZENATTO
JULIA POSSEBON LEME
MARIANA CHAVES RIBEIRO

**SISTEMA WEB DE COLETA, ANÁLISE E APRESENTAÇÃO DE DADOS
SOBRE INTELIGÊNCIA ARTIFICIAL**

**SANTO ANDRÉ
2026**

CAIO YURI LIMA
EDUARDO ZANETTI LUIS
ISABELA TRÓPICO PUZENATTO
JULIA POSSEBON LEME
MARIANA CHAVES RIBEIRO

**SISTEMA WEB DE COLETA, ANÁLISE E APRESENTAÇÃO DE DADOS
SOBRE INTELIGÊNCIA ARTIFICIAL**

Trabalho de conclusão apresentado como
requisito avaliativo da disciplina de
Desenvolvimento de Sistemas, sob orientação
dos Professores Paulo e Raul.

SANTO ANDRÉ
2026

RESUMO

O presente trabalho descreve o desenvolvimento de um sistema web voltado à coleta, armazenamento, organização, consulta e apresentação de dados relacionados ao tema Inteligência Artificial (IA). O sistema foi construído utilizando tecnologias modernas do ecossistema JavaScript, incluindo Node.js, Express.js, Axios, Cheerio, SQLite e geração de relatórios em PDF utilizando uma biblioteca. A interface gráfica, implementada com HTML, CSS e manipulação do DOM, apresenta um dashboard interativo que exibe os dados coletados de páginas da internet de forma organizada e visualmente acessível. A metodologia ágil SCRUM foi adotada para o gerenciamento do projeto, com definição de papéis, backlogs e sprints. Os resultados demonstram a viabilidade e a eficiência da solução proposta para a coleta automatizada de informações sobre IA disponíveis na web.

Palavras-chave: Inteligência Artificial. Web Scraping. Node.js. SQLite. Dashboard. SCRUM.

ABSTRACT

This paper describes the development of a web-based system focused on the collection, storage, organization, retrieval, and presentation of data related to the field of Artificial Intelligence (AI). The system was developed using modern technologies from the JavaScript ecosystem, including Node.js, Express.js, Axios, Cheerio, SQLite, and a PDF report generation library. The graphical interface, implemented with HTML, CSS, and DOM manipulation, features an interactive dashboard that displays data collected from web pages in an organized and visually accessible manner. The SCRUM agile methodology was adopted for project management, including the definition of roles, backlogs, and sprints. The results demonstrate the feasibility and efficiency of the proposed solution for the automated collection of AI-related information available on the web.

Keywords: Artificial Intelligence. Web Scraping. Node.js. SQLite. Dashboard. SCRUM.

LISTA DE ABREVIATURAS E SIGLAS

API – Application Programming Interface

CORS – Cross-Origin Resource Sharing

CSS – Cascading Style Sheets

DOM – Document Object Model

HTML – HyperText Markup Language

HTTP – HyperText Transfer Protocol

IA – Inteligência Artificial

JSON – JavaScript Object Notation

NPM – Node Package Manager

PDF – Portable Document Format

SQL – Structured Query Language

URL – Uniform Resource Locator

SUMÁRIO

1 INTRODUÇÃO.....	7
2 REFERENCIAL TEÓRICO.....	8
2.1 Inteligência Artificial.....	8
2.2 Web Scraping.....	8
2.3 Tecnologias Utilizadas.....	9
3 METODOLOGIA SCRUM.....	10
3.1 Papéis do Time SCRUM.....	10
3.2 Product Backlog.....	10
3.3 Sprint Backlog.....	11
3.4 Sprint Planning.....	12
3.5 Fases da Sprint.....	12
3.6 5W2H.....	13
4 ANÁLISE DE REQUISITOS E FERRAMENTAS DE GESTÃO.....	14
4.1 Análise de Requisitos.....	14
4.1.1 Requisitos Funcionais.....	14
4.1.2 Requisitos Não Funcionais.....	14
5 DESENVOLVIMENTO DO SISTEMA.....	15
5.1 Arquitetura do Sistema.....	15
5.2 Banco de Dados.....	15
5.2.1 Modelo das Tabelas.....	16
5.2.2 Consulta com INNER JOIN.....	16
5.3 Front-end e Dashboard.....	16
5.4 Geração de Relatório PDF.....	17
6 RESULTADOS E DISCUSSÃO.....	18
7 CONCLUSÃO.....	19
REFERÊNCIAS.....	20

1 INTRODUÇÃO

A crescente evolução da Inteligência Artificial (IA) nos últimos anos tem gerado um volume expressivo de informações, artigos, notícias e dados disponíveis na internet. Acompanhar esse fluxo de informações de maneira organizada e eficiente representa um desafio relevante tanto para estudantes quanto para profissionais da área de tecnologia.

Diante desse contexto, o presente trabalho propõe o desenvolvimento de um sistema web capaz de coletar automaticamente dados de páginas relacionadas à Inteligência Artificial, armazená-los em um banco de dados relacional e apresentá-los de forma visual por meio de um dashboard interativo. O sistema permite que o usuário cadastre fontes de pesquisa, inicie a coleta de dados e visualize as informações em relatórios e gráficos.

O projeto foi desenvolvido no âmbito da disciplina de Desenvolvimento de Sistemas, adotando a metodologia ágil SCRUM como base para a organização das atividades. As tecnologias empregadas incluem Node.js, Express.js, Axios, Cheerio, SQLite, entre outras, conforme detalhado nas seções seguintes.

Este documento apresenta a documentação técnica do sistema, estruturada conforme as normas da Associação Brasileira de Normas Técnicas (ABNT), abrangendo referencial teórico, metodologia, desenvolvimento, resultados e considerações finais.

2 REFERENCIAL TEÓRICO

2.1 Inteligência Artificial

A Inteligência Artificial é um campo da ciência da computação que tem como objetivo desenvolver sistemas capazes de realizar tarefas que, quando executadas por seres humanos, requerem inteligência. Segundo Russell e Norvig (2013), IA pode ser definida como o estudo de agentes que recebem percepções do ambiente e executam ações.

Nas últimas décadas, a IA passou por avanços significativos, impulsionados pela disponibilidade de grandes volumes de dados (big data), pelo aumento do poder computacional e pelo desenvolvimento de algoritmos de aprendizado de máquina e aprendizado profundo (deep learning). Aplicações como reconhecimento de voz, processamento de linguagem natural, visão computacional e sistemas de recomendação tornaram-se parte do cotidiano.

A coleta e organização de dados sobre IA disponíveis na web representa, portanto, uma atividade de grande relevância para pesquisadores, estudantes e entusiastas da área, uma vez que permite acompanhar as tendências e avanços mais recentes do campo.

2.1.1 Web Scraping

Web scraping é uma técnica de extração de dados de páginas da internet, realizada de forma automatizada por meio de programas de computador. Essa técnica permite capturar informações estruturadas ou não estruturadas de páginas HTML, transformando-as em dados utilizáveis para análise, armazenamento e apresentação.

De acordo com Mitchell (2018), o web scraping é uma das formas mais eficientes de coleta de dados em larga escala disponíveis atualmente. No contexto do presente trabalho, essa técnica é aplicada para extrair informações relevantes — como títulos, links, imagens e palavras-chave — de páginas relacionadas à Inteligência Artificial.

É importante ressaltar que o uso do web scraping deve respeitar os termos de uso dos sites acessados, bem como as legislações vigentes sobre privacidade e proteção de dados.

2.1.1.1 Tecnologias Utilizadas

A seguir, são descritas as principais tecnologias empregadas no desenvolvimento do sistema:

Node.js: ambiente de execução JavaScript no lado do servidor, baseado no motor V8 do Google Chrome. Permite o desenvolvimento de aplicações de alto desempenho e escaláveis (NODEJS.ORG, 2024).

Express.js: framework minimalista para Node.js, utilizado para a criação de servidores web e APIs RESTful. Facilita o roteamento de requisições HTTP e o gerenciamento de middlewares.

Axios: biblioteca JavaScript para realização de requisições HTTP, tanto no navegador quanto no Node.js. Oferece suporte a Promises e interceptadores de requisição e resposta.

Cheerio: biblioteca para análise e manipulação de HTML no ambiente Node.js, com sintaxe inspirada no jQuery. É amplamente utilizada para tarefas de web scraping.

SQLite: sistema de gerenciamento de banco de dados relacional leve, embutido e sem servidor. Armazena os dados em um único arquivo, sendo ideal para projetos de pequeno e médio porte.

CORS (Cross-Origin Resource Sharing): mecanismo de segurança implementado nos navegadores que controla o acesso a recursos de origens diferentes. No projeto, é configurado no Express.js para permitir a comunicação entre o front-end e o back-end.

3 METODOLOGIA SCRUM

O desenvolvimento do sistema foi organizado com base na metodologia ágil SCRUM, que propõe a divisão do trabalho em ciclos iterativos denominados sprints, com duração definida e entregáveis claros ao final de cada ciclo (SCHWABER; SUTHERLAND, 2020).

3.1 Papéis do Time SCRUM

O time foi organizado conforme os papéis previstos pelo SCRUM:

- Product Owner (PO) - *Mariana Chaves*: responsável por definir e priorizar as funcionalidades do sistema no Product Backlog, representando os interesses do cliente e dos usuários.
- Scrum Master - *Eduardo Zanetti*: responsável por garantir que a equipe siga os princípios e práticas do SCRUM, removendo impedimentos e facilitando os eventos do framework.
- Time de Desenvolvimento - *Restante dos integrantes*: composto pelos demais integrantes do grupo, responsáveis pelo planejamento, desenvolvimento, testes e entrega das funcionalidades.

EQUIPE — RESPONSABILIDADES E DISTRIBUIÇÃO				
Integrante	Papel	Áreas de Atuação	Story Points	Horas Est.
Mariana	Organização SCRUM- Slides Documentação- Diagrama do projeto	Documentação ABNT, Slides, Diagrama, Organização SCRUM, Kanban	20 SP	8h
Caio	Back-end · Scraping · BD	Documentação ABNT, Slides, Prototipagem e CSS.	20 SP	8h
Eduardo	Front e Back-End	SQLite Schema, INNER JOIN, Integração Front- Back, Node.js, Express.js, Axios, Cheerio, Regex, Rotas REST, CORS, SQLite. Slides	21 SP	9H
Julia	Apresentação	Slides.	10 SP	9H

3.1 Product Backlog

PRODUCT BACKLOG										
Projeto: Projeto Desenvolvimento IA Produto Owner: Mariana Chaves Ribeiro Última atualização: 15/05										
ID	Épico	User Story	Critérios de Aceitação	Prioridade	MoSCoW	Story Points	Sprint Alvo	Status	Responsável	Observações
US-001	Interface	Como usuário, quero fazer login com e-mail e senha para acessar minha conta	Tela com nome do projeto, tema IA e menu funcional	Alta	Must Have	3	Sprint 1	A Fazer	Caio Yuri Lima	
US-002	Cadastro	Como usuário, quero cadastrar sites/fontes de pesquisa	Cadastro com nome do site e URL válida	Alta	Must Have	5	Sprint 1	A Fazer	Eduardo Zanetti	
US-003	Coleta de Dados	Como usuário, quero iniciar coleta automática de dados	Sistema acessa site e coleta informações	Alta	Must Have	8	Sprint 1	A Fazer	Eduardo Zanetti	
US-004	Extração HTML	Como usuário, quero extrair títulos, links e imagens das páginas	Dados aparecem corretamente no sistema	Alta	Must Have	8	Sprint 1	A Fazer	Eduardo Zanetti	
US-005	Banco de Dados	Como usuário, quero armazenar os dados coletados no SQLite	Dados gravados corretamente nas tabelas	Alta	Must Have	5	Sprint 1	A Fazer	Eduardo Zanetti	
US-006	Relacionamento	Como usuário, quero consultar dados relacionados	Consulta INNER JOIN funcionando	Alta	Must Have	5	Sprint 1	A Fazer	Eduardo Zanetti	
US-007	API REST	Como usuário, quero cadastrar informações	Rotas POST funcionando corretamente	Alta	Must Have	5	Sprint 1	A Fazer	Eduardo Zanetti	

PRODUCT BACKLOG										
Projeto: Projeto Desenvolvimento IA Produto Owner: Mariana Chaves Ribeiro Última atualização: 15/05										
ID	Épico	User Story	Critérios de Aceitação	Prioridade	MoSCoW	Story Points	Sprint Alvo	Status	Responsável	Observações
US-008	API REST	Como usuário, quero visualizar dados	Rotas GET retornando dados corretamente	Alta	Must Have	8	Sprint 1	A Fazer	Eduardo Zanetti	
US-009	DashBoard	Como usuário, quero visualizar dados em dashboard	Dashboard com gráficos e informações	Alta	Must Have	5	Sprint 1	A Fazer	Eduardo Zanetti	
US-010	PDF	Como usuário, quero gerar relatório PDF	Sistema exporta relatório corretamente	Alta	Must Have	3	Sprint 1	A Fazer	Eduardo Zanetti	
US-011	Admin	Como equipe, queremos organizar tarefas no Kanban	Quadro SCRUM atualizado	Alta	Must Have	2	Sprint 1	A Fazer	Mariana	
US-012	Documentação	Como equipe, queremos documentação ABNT	Documento completo e organizado	Alta	Must Have	5	Sprint 1	A Fazer	Caio Yuri Lima	
US-013	Diagrama	Como equipe, queremos criar diagrama do sistema	Fluxo do sistema representado visualmente	Média	Should Have	2	Sprint 1	A Fazer	Mariana	
US-014	Teste	Como equipe, queremos validar funcionamento do sistema	Testes realizados sem erros graves	Média	Should Have	3	Sprint 1	A Fazer	Equipe	
US-015	Apresentação	Como equipe, queremos criar slides para apresentar o projeto	Slides organizados com tema, tecnologias, funcionamento e conclusão	Média	Should Have	3	Sprint 1	A Fazer	Mariana e Julia	
US-016	Protótipo	Como equipe, queremos desenvolver um protótipo visual do sistema	Protótipo contendo tela inicial, cadastro e dashboard	Alta	Must Have	4	Sprint 1	A Fazer	Caio e Julia	

3.1.1.1 Sprint Backlog

SPRINT BACKLOG									
Sprint Nº: 1 Meta: Desenvolver a estrutura principal do sistema, coleta de dados e documentação para apresentação final. Início: 14/05 Fim: 15/05 Velocidade planejada: Rápido									
ID	User Story	Tarefa	Tipo	Story Points	Estimativa (h)	Responsável	Status	% Concluído	Impedimentos
US-001	Como usuário, quero visualizar uma tela inicial para acessar o sistema facilmente.	Criar tela inicial HTML	Frontend	2	1	Eduardo	Concluído	100%	
US-001	Como usuário, quero cadastrar sites para realizar coleta de dados.	Desenvolver formulário e validação de URL com Regex	Frontend/Backend	3	1	Eduardo	Concluído	100%	
US-001	Como usuário, quero coletar dados automaticamente de páginas web.	Configurar Axios e Cheerio para coleta e extração de informações	Backend	3	5 minutos	Eduardo	Concluído	100%	
US-002	Como usuário, quero armazenar dados coletados para realizar consultas posteriormente.	Criar tabelas SQLite e consultas com INNER JOIN	Banco de Dados	5	5 minutos	Eduardo	Concluído	100%	
US-003	Como usuário, quero consultar informações através do sistema.	Desenvolver rotas GET e POST no Express.js	Backend	2	1	Eduardo	Concluído	100%	
US-003	Como usuário, quero visualizar os dados em um dashboard interativo.	Criar dashboard e manipulação DOM	Frontend	2	1	Eduardo	Concluído	100%	
US-008	Como usuário, quero gerar relatório PDF com os dados coletados.	Implementar geração de PDF no sistema	Backend	3	1	Eduardo	Concluído	100%	

SPRINT BACKLOG									
Sprint Nº: 1 Meta: Desenvolver a estrutura principal do sistema, coleta de dados e documentação para apresentação final. Início: 14/05 Fim: 15/05 Velocidade planejada: Rápido									
ID	User Story	Tarefa	Tipo	Story Points	Estimativa (h)	Responsável	Status	% Concluído	Impedimentos
Informações gerais do sistema. Expressjs									
US-004	Como usuário, quero visualizar os dados em um dashboard interativo.	Criar dashboard e manipulação DOM	Frontend	2	1	Eduardo	Concluído	100%	
US-005	Como usuário, quero gerar relatório PDF com os dados coletados.	Implementar geração de PDF no sistema	Backend	3	1	Eduardo	Concluído	100%	
US-006	Como equipe, queremos organizar o projeto utilizando SCRUM.	Organizar Kanban, Sprint e divisão das tarefas	Gestão	3	1	Mariana	Concluído	100%	
US-007	Como equipe, queremos documentar o projeto para entrega final.	Criar documentação ABNT, diagrama e slides	Documentação	3	2	Caio, Julia e Mariana	Concluído	100%	
US-008	Como usuário, quero visualizar um protótipo do sistema antes da versão final.	Criar protótipo visual do sistema	UX/UI	3	2	Caio e Julia	Concluído	100%	
TOTAL SPRINT				13	10				

3.1.1.1.1 Sprint Planning

SPRINT PLANNING — SISTEMA WEB COM INTELIGÊNCIA ARTIFICIAL				
Coleta de dados SQLite Dashboard PDF			Scrum Acadêmico · Sprint 1	
User Stories	Story Points	Horas Estimadas	Integrantes	Duração
16	68	10h	3	1 semanas
INFORMAÇÕES DA SPRINT				
Objetivo	Entregar sistema web funcional de coleta de dados sobre IA, com scraping (Axios+Cheerio), banco SQLite, INNER JOIN,			
Início	14 de Maio de 2025			
Fim	15 de Maio de 2025			
Duração	1 semana			
Velocidade	13 Story Points planejados			
Daily Standup	Quinta e Sexta — 15 minutos			
Sprint Review	15/05/2025 — Demonstração do sistema completo			
Repositório	GitHub			
Kanban	Miro — colunas: Tarefas para cada integrante / Done			

3.1.1.1.1.1 Fases da Sprint

FASES DA SPRINT				
Fase	Período	Foco Principal	Entregáveis	
Fase 1 — Estrutura	14/mai	Setup, prototipagem, documentação e modelagem do banco	ABNT, prototipo e banco de dados.	
Fase 2 — Funcionalidades	15/mai	Implementação das features principais	Sistema funcionando com todos os requisitos	
Fase 3 — Entrega	15/mai	Integração, testes, documentação e apresentação	Testes, Documentação ABNT, Slides, Sprint Review	

3.1.1.1.1.1 5W2H

Dimensão	Pergunta	Descrição / Resposta	Responsável
WHAT (O quê?)	O que será feito?	Desenvolvimento de um sistema web para coleta, armazenamento, análise e apresentação de dados sobre Inteligência Artificial utilizando Node.js, Express, Axios, Cheerio e SQLite.	Equipe
WHY (Por quê?)	Por que será feito?	Para automatizar a coleta de informações sobre IA, organizar os dados em banco e facilitar a visualização através de dashboard e relatório PDF.	Eduardo Zanetti
WHERE (Onde?)	Onde será feito?	O sistema será desenvolvido em computadores da equipe utilizando Visual Studio Code, Node.js, navegador web e SQLite.	Equipe
WHEN (Quando?)	Quando será feito?	Desenvolvimento nos dias 14 e 15/05, com apresentação final no dia 15/05 à tarde.	Equipe
WHO (Quem?)	Quem irá fazer?	Mariana PO: Diagrama do projeto, slides, testes e Metodologia SCRUM. Caio Desenvolvedor: CSS, ABNT e Protótipo no Figma. Eduardo Desenvolvedor: HTML, Node.js, estrutura do sistema e banco de dados. Julia Desenvolvedora: Figma e slides	Equipe
HOW (Como?)	Como será feito?	Utilizando HTML, CSS e JavaScript no front-end; Node.js e Express no back-end; Axios e Cheerio para coleta de dados; SQLite para armazenamento; SCRUM e Kanban para organização.	Mariana e Eduardo
HOW MUCH (Quanto?)	Quanto vai custar?	O projeto não terá custo financeiro, apenas esforço da equipe e uso de softwares gratuitos. Estimativa de aproximadamente 10 a 15 horas de desenvolvimento.	Equipe

4 ANÁLISE DE REQUISITOS E FERRAMENTAS DE GESTÃO

4.1 Análise de Requisitos

4.1.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer ao usuário:

- Tela inicial com nome do projeto e tema escolhido.
- Formulário para cadastro de sites ou fontes de pesquisa.
- Botão para iniciar a coleta de dados.
- Uso do Axios para acessar páginas externas.
- Uso do Cheerio para extrair informações como títulos, links, imagens, palavras-chave ou descrições.
- Uso de Regex para validar URL, filtrar palavras ou tratar textos.
- Banco SQLite com pelo menos três tabelas relacionadas.
- Consulta com INNER JOIN exibindo dados cruzados.
- Rotas POST para cadastro de dados.
- Rotas GET para listagem e consulta.
- Dashboard visual em HTML, CSS e JavaScript.
- Geração de relatório em PDF.
- Documentação do projeto.
- Apresentação final do sistema funcionando.

4.1.2 Requisitos Não Funcionais

- Desempenho: o sistema deve responder às requisições em no máximo 5 segundos.
- Usabilidade: a interface deve ser intuitiva e de fácil navegação.
- Compatibilidade: o sistema deve funcionar nos navegadores Google Chrome e Mozilla Firefox.
- Manutenibilidade: o código deve ser organizado, comentado e de fácil manutenção.

- Segurança: o sistema deve validar todas as entradas do usuário antes de processá-las.

5 DESENVOLVIMENTO DO SISTEMA

5.1 Arquitetura do Sistema

O sistema segue a arquitetura cliente-servidor, com separação clara entre o front-end (interface do usuário) e o back-end (servidor e banco de dados). A comunicação entre as camadas ocorre por meio de requisições HTTP, utilizando os métodos GET e POST.

A figura a seguir apresenta o diagrama simplificado da arquitetura do sistema:

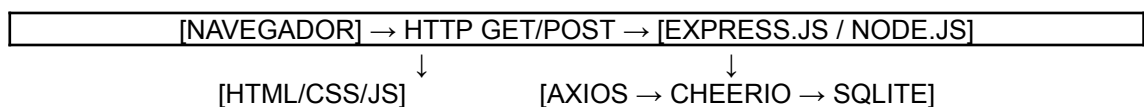


Figura 1 – Diagrama de Arquitetura do Sistema
 Fonte: Elaborado pelos autores (2025)

O fluxo de operação do sistema ocorre da seguinte forma: o usuário acessa a interface web pelo navegador, cadastra uma fonte de pesquisa por meio de um formulário, e inicia a coleta de dados. O servidor Express.js recebe a requisição, utiliza o Axios para acessar a página informada, e o Cheerio para extrair as informações desejadas. Os dados extraídos são então validados por Regex e armazenados no banco de dados SQLite. Por fim, o front-end exibe os dados no dashboard e permite a geração de relatório em PDF.

5.1 Banco de Dados

O banco de dados foi projetado com três tabelas principais, relacionadas entre si por chaves estrangeiras, conforme descrito a seguir:

5.2.1 Modelo das Tabelas

Tabela: temas	Tabela: sites	Tabela: coletas
id_tema (PK)	id_site (PK)	id_coleta (PK)
nome_tema	nome_site	id_site (FK)
descricao	url	titulo_pagina
	id_tema (FK)	quantidade_links
		quantidade_imagens
		palavras_chave
		data_coleta

*Quadro 1 – Modelo de tabelas do banco de dados SQLite
Fonte: Elaborado pelos autores (2025)*

5.2.2 Consulta com INNER JOIN

A principal consulta do sistema utiliza INNER JOIN para cruzar os dados das três tabelas, conforme o exemplo a seguir:

```
SELECT temas.nome_tema, sites.nome_site, sites.url,  
coletas.titulo_pagina, coletas.quantidade_links,  
coletas.data_coleta FROM coletas  
INNER JOIN sites ON coletas.id_site = sites.id_site  
INNER JOIN temas ON sites.id_tema = temas.id_tema;
```

5.3 Front-end e Dashboard

A interface do usuário foi desenvolvida com HTML, CSS e JavaScript puro, sem o uso de frameworks adicionais. O dashboard apresenta os dados coletados em cards informativos, com contadores de links e imagens encontrados, título das páginas analisadas e data da coleta.

A manipulação do DOM é utilizada para atualizar dinamicamente os elementos da página sem necessidade de recarregamento, proporcionando uma experiência de usuário mais fluida. O formulário de cadastro de sites e o botão de início de coleta são os principais elementos de interação disponíveis.

5.4 Geração de Relatório PDF

A geração de relatório em PDF foi implementada utilizando a biblioteca PDFKit (ou similar), que permite criar documentos PDF programaticamente no ambiente Node.js. O relatório inclui cabeçalho com nome do projeto, data de geração, e tabela com os dados das coletas realizadas, organizados por tema e site.

6 RESULTADOS E DISCUSSÃO

O sistema desenvolvido atende a todos os requisitos funcionais definidos na fase de planejamento. Foi possível implementar com sucesso a coleta automatizada de dados de páginas relacionadas à Inteligência Artificial, o armazenamento estruturado em banco de dados SQLite e a apresentação visual por meio do dashboard.

A utilização do Axios em conjunto com o Cheerio demonstrou-se eficiente para a extração de informações das páginas acessadas, permitindo identificar títulos, links, imagens e palavras-chave de forma rápida e confiável. A validação por Regex contribuiu para a integridade dos dados armazenados, impedindo o cadastro de URLs mal formadas.

O banco de dados SQLite, com o relacionamento entre as tabelas temas, sites e coletas, possibilitou consultas ricas por meio do INNER JOIN, cruzando informações de diferentes fontes em uma única query. Isso permitiu a geração de relatórios completos e contextualizados.

Entre as dificuldades encontradas, destacam-se: a configuração inicial do CORS para comunicação entre front-end e back-end; o tratamento de páginas com proteção anti-scraping; e a formatação adequada do relatório PDF. Todas essas dificuldades foram superadas ao longo do desenvolvimento.

A metodologia SCRUM mostrou-se adequada ao contexto do projeto, permitindo a organização eficiente das atividades em um prazo reduzido. A divisão clara de papéis e a realização das sprints contribuíram para a entrega de um produto funcional dentro do tempo disponível.

7 CONCLUSÃO

O presente trabalho demonstrou a viabilidade do desenvolvimento de um sistema web completo para coleta, armazenamento e apresentação de dados sobre Inteligência Artificial, utilizando tecnologias modernas do ecossistema JavaScript e metodologia ágil SCRUM.

O sistema desenvolvido cumpre os objetivos propostos, integrando de forma coesa as tecnologias obrigatórias — Node.js, Express.js, Axios, Cheerio, SQLite, CORS, DOM, Regex e geração de PDF — em uma solução funcional e bem documentada.

Como melhorias futuras, sugere-se: a implementação de autenticação de usuários; a expansão do banco de dados para suportar mais temas e categorias; o desenvolvimento de gráficos e visualizações mais avançadas no dashboard; a adição de agendamento automático de coletas periódicas; e a possibilidade de exportação dos dados em formatos adicionais, como CSV e JSON.

O projeto contribuiu significativamente para o aprendizado prático das tecnologias estudadas em sala de aula, além de desenvolver competências de trabalho em equipe, planejamento e resolução de problemas reais de desenvolvimento de software.

REFERÊNCIAS

AXIOS. Axios Documentation. Disponível em: <https://axios-http.com/docs/intro>. Acesso em: 14 maio 2025.

CHEERIO. Cheerio Documentation. Disponível em: <https://cheerio.js.org/>. Acesso em: 14 maio 2025.

EXPRESS.JS. Express – Node.js web application framework. Disponível em: <https://expressjs.com/>. Acesso em: 14 maio 2025.

MITCHELL, Ryan. Web scraping with Python: collecting more data from the modern web. 2. ed. Sebastopol: O'Reilly Media, 2018.

NODEJS.ORG. Node.js Documentation. Disponível em: <https://nodejs.org/en/docs/>. Acesso em: 14 maio 2025.

RUSSELL, Stuart J.; NORVIG, Peter. Inteligência artificial. 3. ed. Rio de Janeiro: Elsevier, 2013.

SCHWABER, Ken; SUTHERLAND, Jeff. O guia do Scrum: o guia definitivo para o Scrum: as regras do jogo. 2020. Disponível em: <https://scrumguides.org/>. Acesso em: 14 maio 2025.

SQLITE. SQLite Documentation. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 14 maio 2025.

